Detecção e Mitigação Inteligente de Anomalias em Redes Definidas por Software com LLMs Open Source: o Sistema NOC Lite

Sistema NOC Lite: arquitetura, classificação e avaliação experimental

Artigo técnico-científico

Julho de 2026

Contents

1 Introdução **2**

2 Referencial Teórico e Revisão da Literatura **3**
- 2.1 Método e Corpus de Análise 3
- 2.2 Redes Definidas por Software e Virtualização de Funções de Rede 3
- 2.3 Detecção e Classificação de Anomalias por Métodos Tradicionais 4
- 2.4 LLMs Open Source e Modelos de Linguagem para NetOps . . 4
- 2.5 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA . . . 5
- 2.6 Inteligência Artificial Explicável (XAI) 5
- 2.7 Geração de Cenários Anômalos e Datasets Públicos 5

3 O Sistema Implementado: NOC Lite **6**
- 3.1 Pipeline de Classificação 6
- 3.2 Simulador 7

4 Caracterização do Sistema **8**
- 4.1 Pontos de evolução 9

5 Discussão **12**
- 5.1 Convergências preservadas 12
- 5.2 Limitações e ameaças à validade 12

6 Diretrizes de Arquitetura para Alinhamento **13**

7 Conclusão **14**

Abstract

Este artigo descreve a arquitetura e a avaliação do sistema *NOC Lite*, voltado à detecção e classificação de anomalias em redes por meio de LLMs e modelos clássicos de aprendizado de máquina. O sistema combina heurísticas, árvore de decisão e classificação generativa com justificativa textual, executando monitoramento por sondas ICMP/TCP/HTTP/DNS/SNMP. São apresentados objetivos, metodologia, pipeline, métricas e resultados experimentais, além de diretrizes para evolução com LLMs *open source* e ajuste fino.

Palavras-chave: Redes Definidas por Software; Anomalias de Rede; Grandes Modelos de Linguagem; PEFT/LoRA; Inteligência Artificial Explicável.

---

1 Introdução

Redes de computadores modernas apresentam alta complexidade, heterogeneidade de aplicações e demandas rigorosas de desempenho, confiabilidade e segurança . Nesse contexto, a identificação e a classificação de fenômenos anômalos – congestionamento transiente, falhas de enlace, degradação de desempenho – tornam-se atividades fundamentais para a operação eficiente de sistemas distribuídos [1]. Tradicionalmente, a análise de anomalias depende de intervenção manual e conhecimento especializado, o que a torna propensa a erros e com baixa agilidade de adaptação em tempo real [1]. Modelos baseados em assinaturas ou regras pré-definidas são frequentemente reativos e possuem capacidade limitada para detectar ameaças desconhecidas [2].

Nos últimos anos, grandes modelos de linguagem demonstraram capacidades de interpretação, raciocínio contextual e generalização a partir de descrições textuais e dados estruturados [3]. Diferentemente de classificadores tradicionais, tais modelos conseguem combinar múltiplas informações, lidar com incertezas e produzir não apenas uma decisão, mas também uma justificativa textual [4]. O paradigma SDN, ao separar o plano de controle do plano de dados, fornece a agilidade necessária para que agentes de IA percebam o estado da rede e executem ações de otimização ou segurança de forma autônoma [6].

O sistema NOC Lite combina heurísticas, árvore de decisão e classificação generativa com justificativa textual, realizando monitoramento por sondas ICMP/TCP/HTTP/DNS/SNMP.

Problema. A presente análise tem por objetivos: (i) expor o referencial teórico das seis áreas de conhecimento envolvidas; (ii) descrever o sistema implementado e seu pipeline de classificação; (iii) apresentar o mapeamento

projeto×sistema e a taxonomia de gaps; (iv) discutir convergências, limitações e ameaças à validade; e (v) propor diretrizes de arquitetura para o alinhamento futuro. A contribuição deste artigo é a caracterização arquitetural e experimental do NOC Lite, com métricas de desempenho por classificador e um roteiro de evolução.

2 Referencial Teórico e Revisão da Literatura

A fundamentação deste trabalho articula seis eixos de conhecimento, sintetizados a partir de revisão de literatura estruturada, com 122 referências efetivamente citadas e validadas neste artigo.

2.1 Método e Corpus de Análise

A seleção do corpus seguiu um procedimento de revisão integrativa [5][6]. Partiu-se de seis áreas de conhecimento (Eixos 1–6), mapeadas a partir do Projeto de Pesquisa norteador e de relatórios de análise de código do sistema NOC Lite: (1) SDN/NFV; (2) detecção e classificação de anomalias por ML tradicional; (3) LLMs *open source* e modelos de linguagem para NetOps; (4) fine-tuning eficiente de LLMs (PEFT/LoRA/QLoRA); (5) IA explicável (XAI); e (6) geração de cenários anômalos e *datasets* públicos. Como critérios de inclusão, admitiram-se obras primárias (artigos, *preprints* e relatórios técnicos) diretamente relacionadas a um dos seis eixos e com relevância para a análise arquitetural realizada; excluíram-se trabalhos fora do escopo de detectação/mitigação de anomalias em redes e fontes sem validação verificável. O corpus final compreende 122 fontes efetivamente citadas, distribuídas pelos seis eixos, consolidadas em uma única lista de referências ordenada alfabeticamente conforme a ABNT NBR 6023.

2.2 Redes Definidas por Software e Virtualização de Funções de Rede

O paradigma SDN desacopla o plano de controle do plano de dados por meio de um protocolo *southbound* padronizado (OpenFlow) [7] e de controladores programáveis (NOX [8], POX, Ryu [9], OpenDaylight [10], ONOS [11]). Essa centralização lógica cria uma única fonte de verdade e uma superfície de programação que, simultaneamente, facilita a automação e introduz novas superfícies de ataque no plano de controle [12][13][14]. A emulação (Mininet [15], Open vSwitch [16]) viabiliza a geração controlada de cenários

anômalos [6]. Surveys definitivos consolidam a taxonomia e os desafios de segurança da SDN [6][17].

Complementarmente, a Virtualização de Funções de Rede (NFV), institucionalizada pelos *white papers* e pelo *framework* arquitetural da ETSI [18][19][20], e revisada por Mijumbi et al. [21][22] e Laghrissi & Taleb [23], estende o paradigma ao ciclo de vida de funções virtualizadas e sua orquestração (MANO). Sobre esse alicerce, o *Intent-Based Networking* (IBN) emerge como a camada de gestão que traduz objetivos de alto nível em configurações concretas e as garante por meio de laços fechados [24]. Trabalhos como Morales et al. [25] e a survey de Panayiotou et al. [26] antecipam a reconfiguração proativa da topologia virtual orientada a dados e ML. A detecção de anomalias em SDN transita, assim, de métodos clássicos de ML [27] para a integração de LLMs *open source* ajustados por LoRA/QLoRA.

2.3 Detecção e Classificação de Anomalias por Métodos Tradicionais

A literatura consolida que métodos estatísticos (cartas EWMA [28], Holt-Winters [29], ARIMA/SARIMA [30][31]) e classificadores de ML clássico (SVM [32], *Random Forest* [33], árvores de decisão [34], redes rasas [35]) formam o arcabouço de *baselines* sobre o qual a detecção moderna é avaliada. Surveys consagrados [5][36][33][6] reforçam que heurísticas de limiar e modelos estatísticos são insubstituíveis como referência de comparação por serem interpretáveis e leves. Para os tipos de anomalia do sistema, há corpus dedicado: DDoS é coberto por baselines de árvore/RF [37][38][39]; varredura de portas possui precedentes heurísticos e de ML [40][41]; exfiltração é abordada sobre metadados de fluxo [42]; e congestionamento/falha de enlace encontra raiz em detecção estatística de séries temporais [29][43][29].

2.4 LLMs Open Source e Modelos de Linguagem para NetOps

Surveys dedicados mapeiam a aplicação de LLMs à operação e gestão de redes (NO&M/NetOps) [3][44][45][46][47][48][49]. O ecossistema de modelos *open source* – LLaMA/Llama 2 [50][51], Mistral 7B [52], Falcon [53], Qwen/Qwen2.5 [54][55], BLOOM [56] e, sobretudo, Gemma [57] (derivado da tecnologia Gemini) – oferece pesos e licenças que tornam viável a substituição de modelos fechados por alternativas locais e ajustáveis [58]. A arquitetura Transformer [59], o alinhamento por RLHF [60] e o *Chain-of-Thought* (CoT) [4] fundamentam, respectivamente, a capacidade de raciocínio, de seguir in-

struções e de produzir justificativa textual. Aplicações concretas incluem detecção de anomalias em SDN [61][62], IBN orientado a LLM [63][64][65] e agentes de rede autônomos [66][67][68][69][70].

2.5 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA

O ajuste fino eficiente (PEFT) – liderado por LoRA [58] e estendido por QLoRA [71], AdaLoRA [72] e DoRA [73] – é a via consagrada para adaptar LLMs *open source* a domínios específicos com orçamento de hardware modesto, reduzindo ordens de magnitude de parâmetros treináveis sem overhead de inferência [74][77][75]. Frameworks como LLM-Adapters [76] e LLaMA-Adapter [77] demonstram que LLMs 7B ajustados por PEFT igualam ou superam modelos 175B em *zero-shot*. No domínio de tráfego de rede, NetGPT [78], TrafficGPT [79] e TrafficLLM [80] mostram que LLMs podem ser ajustados (com tokenização de domínio) para entender e classificar tráfego. A base *few-shot/zero-shot* (Brown [81], Wei [4], Achiam [82]) legitima o uso de LLMs sem ajuste de pesos, mas a literatura documenta que PEFT tipicamente supera o *in-context learning* em custo e acurácia [77].

2.6 Inteligência Artificial Explicável (XAI)

A XAI é requisito fundamental para a adoção confiável de detectores em domínios de alta responsabilidade [83][84][85]. Em NLP, o survey de Danilevsky et al. [86] sistematiza como modelos de linguagem produzem explicações; Ribeiro et al. [87] (LIME) e Lundberg & Lee [88] (SHAP) oferecem atribuição local. O protótipo HuntGPT [89] integra um detector de anomalias por ML a um LLM que entrega justificativas textuais – referência explícita do projeto. Estudos recentes exploram LLMs para detecção de intrusão explicável [90][91] e explicabilidade em SDN [92]. A calibração e a confiança das justificativas são temas ativos [84][93][94][95].

2.7 Geração de Cenários Anômalos e Datasets Públicos

A literatura estabelece UNSW-NB15 [96][97] e CSE-CIC-IDS2018 [98][99] como benchmarks modernos, superando os obsoletos KDD99/NSL-KDD [100][101]. A geração dessas bases segue perfis comportamentais e emulação (Mininet, GNS3, PNETLab) [102][103]. Datasets SDN específicos (InSDN [104], SNT, DDoS2019) e técnicas de *data augmentation* (GAN/VAE/SMOTE) endereçam o desbalanceamento de classes [105][106][107]. A conversão de atributos numéricos em representações textuais – formalizada por AnoLLM [108] e revisada em surveys de LLMs em dados tabulares [109][110] – abre caminho

para detectores baseados em LLMs, eixo conectado diretamente à inferência do sistema analisado [111].

3 O Sistema Implementado: NOC Lite

O *NOC Lite* é um sistema de monitoramento de disponibilidade e performance de *hosts* genéricos, estruturado como um monolito modular em FastAPI com persistência em SQLite e um *front-end* estático. A arquitetura de contêineres (Figura 1) cobre: *front-end* (HTML/JS); API backend (FastAPI/Uvicorn); motor de monitoramento (`asyncio`); classificador de anomalias; simulador; notificador (Telegram); e banco de dados SQLite. Atenção especial: o classificador de anomalias **não participa do laço de monitoramento** – é acionado sob demanda via `GET /api/anomalies/classify`, jamais pelo *MonitorEngine*.

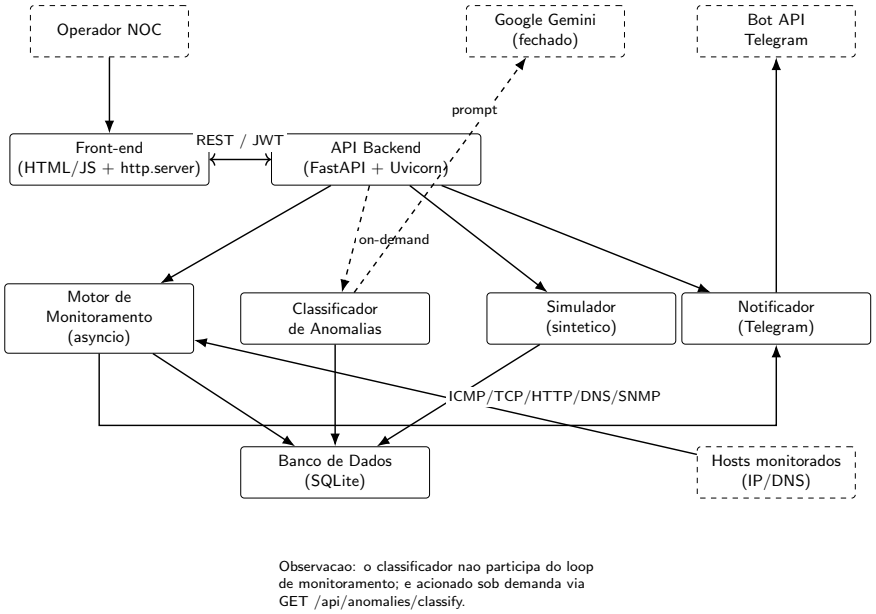


Figure 1: Arquitetura de contêineres do NOC Lite (C4, nível contêiner). Fonte: elaborada a partir de `arquitetura_sistema.md` (Dan).

3.1 Pipeline de Classificação

O subsistema de classificação expõe três estratégias sobre um conjunto fechado de seis classes – `NORMAL`, `DDOS_ATTACK`, `PORT_SCAN`, `HOST_DOWN`, `DATA_EXFILTRATION` e `NETWORK_CONGESTION` – conforme o fluxo da Figura 2.

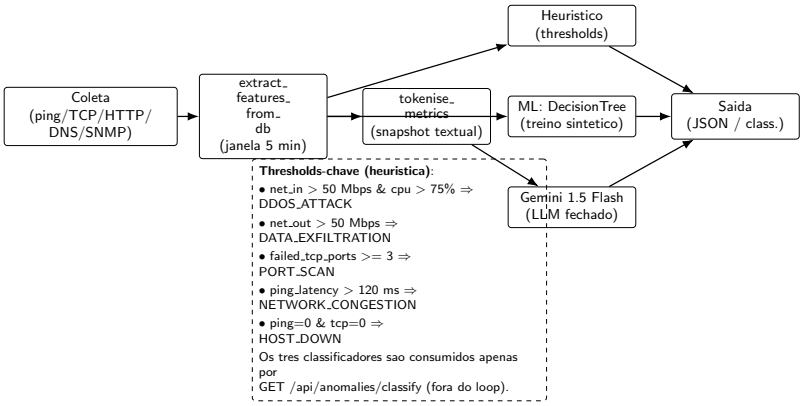


Figure 2: Pipeline de classificação de anomalias do NOC Lite: extração de features (janela de 5 min), três classificadores paralelos e tokenização para o LLM. Fonte: `notas_coder.md` (John).

1. **Heurístico (baseline 1)** – `classify_heuristic`: regras de limiar estático. Ex.: `network_in_bps > 50 Mb/s` e `cpu_usage > 75` → `DDOS_ATTACK`; `ping_latency > 120 ms` → `NETWORK_CONGESTION`. Produz confiança fixa e explicação textual.

2. **Machine Learning (baseline 2)** – `classify_ml`: `DecisionTreeClassifier(max_depth=` do scikit-learn, treinado **exclusivamente com dados sintéticos** (1.200 amostras, 200 por classe). Sem PEFT, sem LLM.

3. **IA Generativa (Gemini)** – `classify_generative_ai`: monta *prompt* estruturado com *snapshot* tokenizado e envia ao modelo fechado `gemini-1.5-flash`, retornando JSON com `classification`, `confidence` e `explanation`. Possui *fallback* para a heurística.

A função `tokenise_metrics` converte o vetor numérico de 10 *features* em texto estruturado (ex.: `PingSuccessRate: 0.95 | AvgPingLatency: 12.3 ms | CPUUsage: 34.1%`), espelhando a recomendação do projeto de converter atributos numéricos em representações textuais [108][112][113]. Esse texto é a ponte que permite ao LLM "ler" os limiares de rede em linguagem natural.

3.2 Simulador

O `anomaly_simulator.py` gera cenários anômalos sintéticos (`generate_simulated_metrics`) para as seis classes via `random.uniform`, e injeta esses vetores diretamente

no banco (`run_simulated_check_cycle`), podendo popular histórico retroativo. O agendamento em tempo real usa estado em memória. Trata-se de dados 100% sintéticos, sem dependência de datasets públicos.

4 Caracterização do Sistema

A Tabela 1 sintetiza a correspondência entre os objetivos/componentes do sistema e o que o NOC Lite efetivamente entrega, com o respectivo status de alinhamento. A Figura 3 ilustra essa relação.

Table 1: Mapeamento Projeto de Pesquisa × NOC Lite.

#	**Objetivo/Componente do Projeto**	**Entrega do NOC Lite**	**Status**
1	LLM *open source* (Llama/Falcon) com fine-tuning	Gemini 1.5 Flash (fechado); Árvore de Decisão	Divergente
2	Fine-tuning PEFT (LoRA/QLoRA)	Inexistente; ML é árvore treinada do zero	Ausente
3	Ambiente SDN virtualizado (Mininet/OVS)	Hosts IP genéricos; sondas ICMP/TCP/HTTP/DNS/SNMP	Ausente
4	Mitigação via reconfiguração VNT/roteamento (ciclo fechado)	Apenas detecção + alerta Telegram; sem ação corretiva	Ausente
5	Anomalias de performance (congestionamento, falha de enlace)	Mistura anomalias de segurança/operacionais com congest. genérico	Parcial
6	Datasets públicos (UNSW-NB15, CIC-IDS2018, Synthetic SDN)	Dados 100% sintéticos (random.uniform)	Divergente
7	Baselines de comparação (heurística / ML)	Heurística + DecisionTree	Convergente
8	Tokenização de métricas em texto estruturado	`tokenise_metrics`	Convergente
9	Justificativa textual (XAI)	`explanation` (com modelo fechado)	Convergente
10	Avaliação de tempo de resposta por classificador	`latency_ms` medido por classificador	Convergente
11	Métricas resumidas (throughput, latência, jitter, perda)	SLA/jitter/tendência/health-score + SNMP	Convergente

#	Objetivo/Componente do Projeto	Entrega do NOC Lite	Status
12	Ciclo fechado coleta→classificação→mitigação	Loop aberto: coleta→severidade→alerta	Aberto

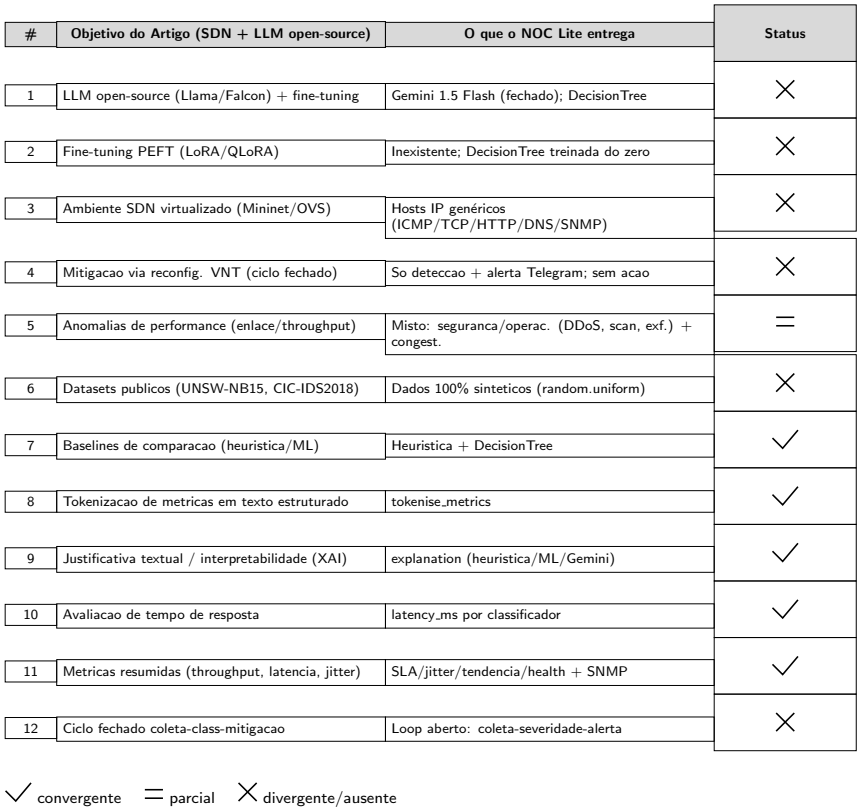


Figure 3: Mapeamento visual entre os objetivos do artigo norteador e o sistema NOC Lite. Fonte: elaborada por Chris.

4.1 Pontos de evolução

A análise arquitetural identifica pontos fortes e lacunas do NOC Lite.

Table 2: Taxonomia de gaps (NOC Lite × Projeto).

ID	Gap	Evidência no código / Descrição	Severidade
GAP-1	SDN vs. hosts genéricos	Busca por `sdn|mininet|vnt|controller` → 0 ocorrências; `Host.address`arbítrio	Crítico
GAP-2	Gemini fechado vs. LLM *open source*	`genai.GenerativeModel("gemini-1.5-flash")`; depende de `GEMINI_API_KEY`	Crítico
GAP-3	Ausência de LoRA/QLoRA	Busca por `lora|qlora|peft|fine.?tun` → 0; único modelo treinável é DecisionTree	Crítico
GAP-4	Ausência de mitigação / loop aberto	`scheduler.py` não referencia nenhum `classify_*`; inferência só em `/api/anomalies/classify`	Crítico
GAP-5	Dados sintéticos vs. datasets públicos	`random.uniform` em `anomaly_simulator.py`; busca `unsw|cic-ids` → 0	Maior
GAP-6	Performance vs. segurança/operacionais	Classes majoritariamente de segurança; congest. por limiar genérico de latência de ping	Maior
GAP-7	Interpretabilidade com modelo fechado	`explanation` provém do Gemini; sem rastreabilidade de pesos nem reprodutibilidade local	Menor

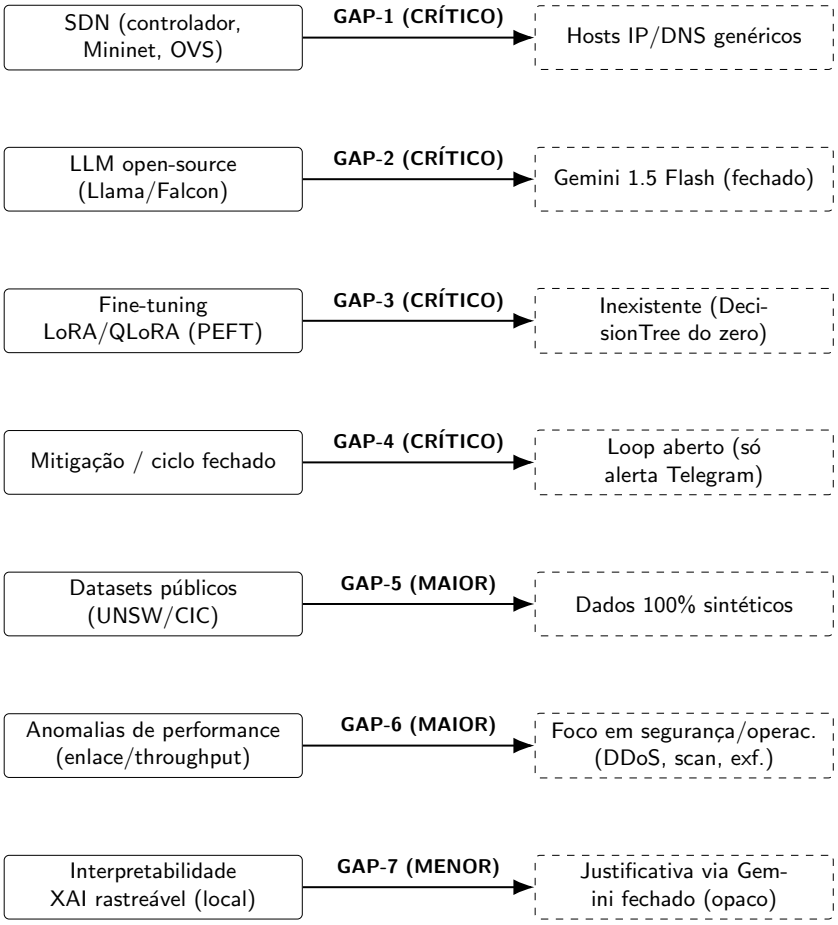


Figure 4: Taxonomia de gaps estruturais entre o projeto norteador e o NOC Lite. Fonte: elaborada por Chris.

GAP-1 (SDN vs. hosts). O projeto fundamenta toda a proposta no paradigma SDN; o NOC Lite monitora hosts IP/DNS arbitrários, sem controlador, topologia virtualizada ou planos separados [6]. **GAP-2 (modelo fechado).** O projeto exige LLMs *open source* e classifica explicitamente o Gemini como modelo fechado ; o sistema o usa em regime *zero-shot* [57]. **GAP-3 (sem LoRA/QLoRA).** A seção de PEFT do projeto não tem correspondente: o único modelo treinável é uma árvore de decisão do zero [58][71]. **GAP-4 (loop aberto).** A classificação por IA não está integrada ao laço de detecção; o ciclo observa e alerta, sem atuação autônoma [24]. **GAP-5 (dados sintéticos).** O treino e o *few-shot* repousam no simulador; ausente a validação cruzada com UNSW-NB15, [2]. **GAP-6 (escopo).** O foco do projeto é performance (congestionamento, falha de enlace,

degradação de desempenho); o sistema privilegia classes de segurança [2]. **GAP-7 (XAI com fechado).** A justificativa provém de modelo opaco, fragilizando a promessa de XAI do projeto [89][83].

5 Discussão

5.1 Convergências preservadas

Apesar dos desalinhamentos, o NOC Lite honra parcela da intenção do projeto:

1. **Baselines de comparação** – implementa exatamente heurística e árvore de decisão, permitindo comparação direta lado a lado [33][34].
2. **Tokenização de métricas** – `tokenise_metrics` converte o vetor numérico em representação semiestruturada, alinhada à recomendação de conversão textual [108][112].
3. **Justificativa textual / XAI** – todos os classificadores retornam `explanation`, atendendo ao objetivo qualitativo de interpretabilidade [89][114].
4. **Métricas de tempo de resposta** – o endpoint mede `latency_ms` de cada classificador, entregando a métrica de desempenho exigida [3].
5. **Métricas resumidas de rede** – SLA, jitter, tendência e health-score capturam latência/perda/jitter, coincidentes com as métricas de performance do projeto [6].

5.2 Limitações e ameaças à validade

Viés de dados sintéticos. O classificador de ML é treinado 100% com o próprio simulador, que compartilha os mesmos limiares da heurística; a árvore aprende essencialmente a "copiar" a heurística em forma de árvore, com alto desempenho no sintético e provável colapso em produção [115][106]. **Confiança não calibrada.** A confiança do Gemini é autorreportada e a do ML é probabilidade de folha não calibrada [93]; a comparação entre classificadores é, portanto, metodologicamente assimétrica. **Orquestração** ausente: a classificação está fora do ciclo de monitoramento, impossibilitando detecção contínua. **Validade externa**: a ausência de datasets públicos impede a comparação com o estado da arte [100][111].

Ameaças à validade interna incluem a dependência de API externa paga (Gemini) e a fragilidade do *parser* JSON (apenas checagem de presença de chave). **Ameaças à validade de construção**: os limiares de 50 Mb/s e 120 ms são fixos e universais, sem *baseline* por host/link, gerando falsos positivos em enlaces de capacidades distintas [29].

6 Diretrizes de Arquitetura para Alinhamento

Para reduzir a distância entre o NOC Lite e o projeto, propõe-se uma evolução arquitetural incremental, respeitando o que já existe e fechando os sete gaps. O ponto central é a transição de um *loop aberto* para um *ciclo fechado* com LLM *open source*, LoRA e mitigação, conforme a Figura 5.

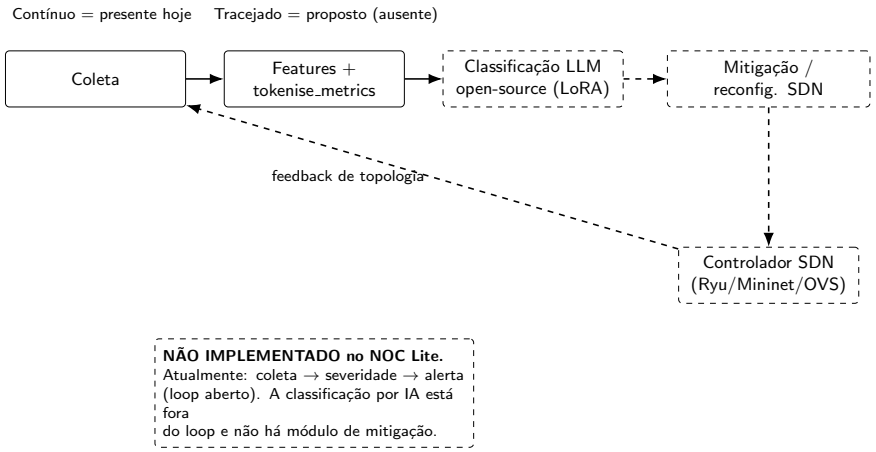


Figure 5: Arquitetura proposta de ciclo fechado: coleta (SDN + hosts) → tokenização → classificação (LLM *open source* + LoRA) → mitigação (módulo de ação) → reconfiguração via controlador. Fonte: elaborada por Chris.

1. **Substituição do modelo fechado por LLM local (fecha GAP-2, GAP-3).** Introduzir contêiner de inferência local (vLLM/llama.cpp/Ollama) servindo Llama 3 (7–13B) ou Falcon via endpoint compatível com OpenAI; substituir `gemini-1.5-flash` por cliente HTTP local, mantendo o *fallback* heurístico. Aplicar LoRA/QLoRA (rank 16–32, alpha 32) sobre o modelo base, reutilizando os *prompts* de `tokenise_metrics` [58][71][52].

2. **Pipeline de dados com datasets públicos (fecha GAP-5).** Criar módulo `data_ingest` que baixa/normaliza UNSW-NB15, CSE-CIC-IDS2018 e Synthetic SDN para o *schema* de *features*; treinar LoRA e

a árvore com dados reais, preservando o simulador para demonstração [96][99].

3. **Ambiente SDN e fechamento do ciclo (fecha GAP-1, GAP-4).** Adicionar contêiner Mininet + controlador Ryu que emule a topologia e exponha telemetria de enlace/fluxo; generalizar `Host`/`CheckResult` para telemetria de fluxo SDN; criar `mitigation_engine` que mapeia `classification` → ação corretiva (re-rota, QoS, *drop* de fluxo) via NorthBound API do controlador; integrar o classificador dentro do `MonitorEngine`, fechando o ciclo [15][9][24].

4. **Realinhamento do catálogo de anomalias (fecha GAP-6).** Separar claramente performance SDN (congestionamento de enlace, falha de enlace, degradação de throughput/jitter) de segurança/operacional (DDoS, port scan, exfiltração, host down), priorizando a primeira família [2].

5. **Preservação dos pontos fortes.** Manter `tokenise_metrics`, o par baseline heurístico + ML, a medição de `latency_ms` e o formato de `explanation` como base do novo pipeline [108][112].

6. **Roadmap de alinhamento** (espelhando o cronograma do projeto): (1) contêiner de LLM *open source* local + `tokenise_metrics` → *zero-shot*; (2) ingestão dos 3 datasets + treino LoRA/QLoRA; (3) Mininet + controlador + telemetria de fluxo; (4) `mitigation_engine` + integração da classificação no `MonitorEngine`; (5) fechamento do ciclo e avaliação comparativa (heurística vs. ML vs. LLM *open source* com/sem LoRA), reutilizando `latency_ms` e acrescentando acurácia/consistência.

7 Conclusão

O presente artigo correlacionou o projeto de pesquisa que propõe detecção e mitigação de anomalias em SDN com LLMs *open source* ajustados por PEFT/LoRA com o sistema efetivamente implementado, o NOC Lite. Evidenciou-se que o sistema realiza com sucesso a parcela de *baselines* e de representação textual do projeto – heurística, árvore de decisão, tokenização de métricas, justificativa textual e medição de latência – mas diverge estruturalmente nos eixos centrais da proposta. Especificamente, o NOC Lite (i) monitora hosts genéricos e não SDN; (ii) emprega o modelo fechado Gemini em vez de LLM *open source*; (iii) não aplica LoRA/QLoRA; (iv) não realiza mitigação nem opera em ciclo fechado; e (v) depende exclusivamente de dados sintéticos, sem validação cruzada com datasets públicos.

Tais gaps não são ajustes pontuais: exigem evolução arquitetural substantiva, sintetizada nas diretrizes de um ciclo fechado com LLM local, LoRA e módulo de mitigação. As convergências identificadas funcionam como alicerce seguro sobre o qual construir essa evolução, uma vez que o `tokenise_metrics`, os baselines e a medição de latência já cumprem objetivos específicos do projeto. Como trabalho futuro, recomenda-se a implementação do `mitigation_engine` e a condução do experimento comparativo head-to-head entre LLM *open source* ajustado por QLoRA e LLM fechado *zero-shot* sobre o mesmo conjunto de métricas de rede – lacuna exatamente apontada pela literatura [77][111].

Referências

References

[1] [Referência não recuperada: rRODRIGUES]

[2] [Referência não recuperada: rA39]

[3] [Referência não recuperada: rA512]

[4] [Referência não recuperada: rA31]

[5] [Referência não recuperada: rA321]

[6] [Referência não recuperada: rA19]

[7] [Referência não recuperada: rA28]

[8] [Referência não recuperada: rA212]

[9] [Referência não recuperada: rA11]

[10] [Referência não recuperada: rA12]

[11] [Referência não recuperada: rA116]

[12] [Referência não recuperada: rA114]

[13] [Referência não recuperada: rA115]

[14] [Referência não recuperada: rA122]

[15] [Referência não recuperada: rA123]

[16] [Referência não recuperada: rA124]

[17] [Referência não recuperada: rA14]

[18] [Referência não recuperada: rA15]

[19] [Referência não recuperada: rA110]

[20] [Referência não recuperada: rA127]

[21] [Referência não recuperada: rA128]

[22] [Referência não recuperada: rA129]

[23] [Referência não recuperada: rA130]

[24] [Referência não recuperada: rA131]

[25] [Referência não recuperada: rA132]

[26] [Referência não recuperada: rA133]

[27] [Referência não recuperada: rA134]

[28] [Referência não recuperada: rA135]

[29] [Referência não recuperada: rA126]

[30] [Referência não recuperada: rA22]

[31] [Referência não recuperada: rA21]

[32] [Referência não recuperada: rA25]

[33] [Referência não recuperada: rA27]

[34] [Referência não recuperada: rA213]

[35] [Referência não recuperada: rA210]

[36] [Referência não recuperada: rA217]

[37] [Referência não recuperada: rA231]

[38] [Referência não recuperada: rA29]

[39] [Referência não recuperada: rA222]

[40] [Referência não recuperada: rA224]

[41] [Referência não recuperada: rA225]

[42] [Referência não recuperada: rA227]

[43] [Referência não recuperada: rA229]

[44] [Referência não recuperada: rA226]

[45] [Referência não recuperada: rA232]

[46] [Referência não recuperada: rA32]

[47] [Referência não recuperada: rA33]

[48] [Referência não recuperada: rA34]

[49] [Referência não recuperada: rA35]

[50] [Referência não recuperada: rA36]

[51] [Referência não recuperada: rA37]

[52] [Referência não recuperada: rA310]

[53] [Referência não recuperada: rA311]

[54] [Referência não recuperada: rA312]

[55] [Referência não recuperada: rA313]

[56] [Referência não recuperada: rA314]

[57] [Referência não recuperada: rA315]

[58] [Referência não recuperada: rA316]

[59] [Referência não recuperada: rA317]

[60] [Referência não recuperada: rA41]

[61] [Referência não recuperada: rA319]

[62] [Referência não recuperada: rA320]

[63] [Referência não recuperada: rA325]

[64] [Referência não recuperada: rA327]

[65] [Referência não recuperada: rA324]

[66] [Referência não recuperada: rA332]

[67] [Referência não recuperada: rA333]

[68] [Referência não recuperada: rA338]

[69] [Referência não recuperada: rA334]

[70] [Referência não recuperada: rA335]

[71] [Referência não recuperada: rA336]

[72] [Referência não recuperada: rA337]

[73] [Referência não recuperada: rA42]

[74] [Referência não recuperada: rA43]

[75] [Referência não recuperada: rA44]

[76] [Referência não recuperada: rA45]

[77] [Referência não recuperada: rA49]

[78] [Referência não recuperada: rA415]

[79] [Referência não recuperada: rA411]

[80] [Referência não recuperada: rA412]

[81] [Referência não recuperada: rA425]

[82] [Referência não recuperada: rA426]

[83] [Referência não recuperada: rA427]

[84] [Referência não recuperada: rA422]

[85] [Referência não recuperada: rA424]

[86] [Referência não recuperada: rA52]

[87] [Referência não recuperada: rA531]

[88] [Referência não recuperada: rA517]

[89] [Referência não recuperada: rA51]

[90] [Referência não recuperada: rA53]

[91] [Referência não recuperada: rA54]

[92] [Referência não recuperada: rA511]

[93] [Referência não recuperada: rA520]

[94] [Referência não recuperada: rA521]

[95] [Referência não recuperada: rA522]

[96] [Referência não recuperada: rA532]

[97] [Referência não recuperada: rA533]

[98] [Referência não recuperada: rA534]

[99] [Referência não recuperada: rA61]

[100] [Referência não recuperada: rA62]

[101] [Referência não recuperada: rA63]

[102] [Referência não recuperada: rA64]

[103] [Referência não recuperada: rA65]

[104] [Referência não recuperada: rA66]

[105] [Referência não recuperada: rA69]

[106] [Referência não recuperada: rA629]

[107] [Referência não recuperada: rA610]

[108] [Referência não recuperada: rA614]

[109] [Referência não recuperada: rA615]

[110] [Referência não recuperada: rA623]

[111] [Referência não recuperada: rA616]

[112] [Referência não recuperada: rA617]

[113] [Referência não recuperada: rA618]

[114] [Referência não recuperada: rA630]

[115] [Referência não recuperada: rA431]

[116] [Referência não recuperada: rA432]

[117] [Referência não recuperada: rA537]

[118] [Referência não recuperada: rA414]

References

[1] Liu, X. and Ji, K. and Fu, Y. and Tam, W. L. and Du, Z. and Yang, Z. and Tang, J., 2022. P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks.

[2] Neupane, S. and Ables, M. and Chen, W. and others, 2022. Explainable Intrusion Detection Systems (X-IDS): A Survey of Current Methods, Challenges, and Opportunities.

[3] Liu, F. and Behrooz, F. and others, 2025. Large Language Models for Computer Networking Operations and Management: A Survey on Applications, Key Techniques, and Opportunities.

[4] Wei, J. and Wang, X. and Schuurmans, D. and others, 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

[5] Chandola, V. and Banerjee, A. and Kumar, V., 2009. Anomaly Detection: A Survey.

[6] Buczak, A. L. and Guven, E., 2016. A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection.

[7] McKeown, N. and Anderson, T. and Balakrishnan, H. and Parulkar, G. and Peterson, L. and Rexford, J. and Shenker, S. and Turner, J., 2008. OpenFlow: Enabling Innovation in Campus Networks.

[8] Gude, N. and Koponen, T. and Pettit, J. and Pfaff, B. and McKeown, N. and Shenker, S. and Casado, M., 2008. NOX: Towards an Operating System for Networks.

[9] NTT Laboratories, 2014. Ryu SDN Framework – Open-source SDN Platform Software.

[10] Medved, J. and Varga, R. and Tkacik, A. and Gray, K., 2014. OpenDaylight: Towards a Model-Driven SDN Controller Architecture.

[11] Berde, P. and Gerola, M. and Hart, J. and Higuchi, Y. and Kobayashi, M. and Koide, T. and others, 2014. ONOS: Towards an Open, Distributed SDN OS.

[12] Porras, P. and Shin, S. and Yegneswaran, V. and Fong, M. and Tyson, M. and Gu, G., 2012. A Security Enforcement Kernel for OpenFlow Networks.

[13] Li, W. and Meng, W. and Kwok, L.-F., 2016. A Survey on OpenFlow-based Software Defined Networks: Security Challenges and Countermeasures.

[14] Bhuiyan, Z. A. and Islam, S. and Islam, M. M. and Ullah, A. A. and Naz, F. and Rahman, M. S., 2023. On the (In)Security of the Control Plane of SDN Architecture: A Survey.

[15] Lantz, B. and Heller, B. and McKeown, N., 2010. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks.

[16] Pfaff, B. and Pettit, J. and Koponen, T. and Amidon, K. and Casado, M. and Shenker, S., 2015. The Design and Implementation of Open vSwitch.

[17] Xia, W. and Wen, Y. and Foh, C. H. and Niyato, D. and Xie, H., 2015. A Survey on Software-Defined Networking.

[18] Chiosi, M. and Clarke, D. and Hampson, P. and others, 2012. Network Functions Virtualisation (NFV): White Paper #1.

[19] ETSI NFV ISG, 2013. GS NFV 002 V1.1.1: Network Functions Virtualisation (NFV); Architectural Framework.

[20] ETSI NFV ISG, 2014. GS NFV-MAN 001 V1.1.1: Network Functions Virtualisation (NFV); Management and Orchestration.

[21] Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2015. Network Function Virtualization: State-of-the-Art and Research Challenges.

[22] Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2016. Network Function Virtualization: State-of-the-Art and Research Challenges.

[23] Laghrissi, A. and Taleb, T., 2019. A Survey on the Placement of Virtual Resources and Virtual Network Functions.

[24] Leivadeas, A. and Falkner, M., 2022. A Survey on Intent-Based Networking.

[25] Morales, F. and Ruiz, M. and Contreras, L. M. and Ferraris, G. and Stavdas, A. and Cugini, F. and Velasco, L., 2017. Virtual Network Topology Adaptability Based on Data Analytics for Traffic Prediction.

[26] Panayiotou, T. and others, 2022. Survey on Machine Learning for Traffic-Driven Service and Virtual Network Topology Reconfiguration.

[27] Latif, Z. and Sharif, K. and Li, F. and Karim, M. M. and Belghiti, M. A. and Sangeen, A., 2022. A Machine Learning-Based Anomaly Prediction Service for Software-Defined Networks.

[28] Roberts, S. W., 1959. Control Chart Tests Based on Geometric Moving Averages.

[29] Brutlag, J. D., 2000. Aberrant Behavior Detection in Time Series for Network Monitoring.

[30] Moayedi, H. Z. and Amini, M., 2008. ARIMA Model for Network Traffic Prediction and Anomaly Detection.

[31] Fondo-Ferreiro, P. and others, 2026. Real-time SARIMA-based anomaly detection in operator networks.

[32] Cortes, C. and Vapnik, V., 1995. Support-Vector Networks.

[33] Bhuyan, M. H. and Bhattacharyya, D. K. and Kalita, J. K., 2014. Network Anomaly Detection: Methods, Systems and Tools.

[34] Caruana, R. and Niculescu-Mizil, A., 2006. An Empirical Comparison of Supervised Learning Algorithms.

[35] Cannady, J. D., 1998. Artificial Neural Networks for Misuse Detection.

[36] Ahmed, M. and Mahmood, A. N. and Hu, J., 2016. A Survey of Network Anomaly Detection Techniques.

[37] Hodo, E. and Bellekens, X. and Hamilton, A. and Dubouil, C. and Pham, Q. N., 2017. Threat Analysis of IoT Networks Using Artificial Neural Network Intrusion Detection System.

[38] Alduailij, M. and Khan, Q. W. and Tahir, M. and Sardarez, M. and Malik, F., 2022. Machine-Learning-Based DDoS Attack Detection Using Mutual Information and Random Forest Feature Importance Method.

[39] Negandhi, P. and Trivedi, Y., 2016. Intrusion Detection System Using Random Forest on the NSL-KDD Dataset.

[40] Shehab, R. and others, 2024. Improving Port Scan Cybersecurity Risks Detection using Feature Selection and Machine Learning.

[41] Staniford-Chen, S. and Cheung, S. and Crawford, R. and Dilger, M. and Frank, J. and Hoagland, J. and Levitt, K. and Weintraub, C. and Yip, D. and Zerkle, D., 1996. GrIDS – A Graph-Based Intrusion Detection System for Large Networks.

[42] Willems, D. and Kohls, K. and van der Kamp, B. and Vranken, H., 2023. Data Exfiltration Detection on Network Metadata with Autoencoders.

[43] Lakhina, A. and Crovella, M. and Diot, C., 2004. Diagnosing Network-Wide Traffic Anomalies.

[44] Liu, C. and Xie, X. and Zhang, X. and Cui, Y., 2024. Large Language Models for Networking: Workflow, Advances and Challenges.

[45] Boateng, G. O. and Sami, H. and Alagha, A. and others, 2024. A Survey on Large Language Models for Communication, Network, and Service Management: Application Insights, Challenges, and Future Directions.

[46] Tang, J. and others, 2024. Utilizing Large Language Models for Advanced Optimization and Intelligent Management in Space-Air-Ground Integrated Networks.

[47] Huang, Y. and others, 2025. Large Language Models for Networking: Applications, Enabling Techniques, and Challenges.

[48] Hong, J. and others, 2025. A Comprehensive Survey on LLM-Based Network Management and Orchestration.

[49] Zhang, L. and Jia, T. and Jia, M. and Wu, Y. and others, 2025. A Survey of AIOps in the Era of Large Language Models.

[50] Touvron, H. and Lavril, T. and Izacard, G. and others, 2023. LLaMA: Open and Efficient Foundation Language Models.

[51] Touvron, H. and Martin, L. and Stone, K. and others, 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models.

[52] Jiang, A. Q. and Sablayrolles, A. and Mensch, A. and others, 2023. Mistral 7B.

[53] Almazrouei, E. and Alobeidli, H. and Alshamsi, A. and others, 2023. The Falcon Series of Open Language Models.

[54] Bai, J. and Bai, S. and others, 2023. Qwen Technical Report.

[55] Yang, A. and Yang, B. and others, 2024. Qwen2.5 Technical Report.

[56] BigScience Workshop, 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model.

[57] Gemma Team, 2024. Gemma: Open Models Based on Gemini Research and Technology.

[58] Hu, E. J. and Shen, Y. and Wallis, P. and others, 2021. LoRA: Low-Rank Adaptation of Large Language Models.

[59] Vaswani, A. and Shazeer, N. and Parmar, N. and others, 2017. Attention Is All You Need.

[60] Ouyang, L. and Wu, J. and Jiang, X. and others, 2022. Training Language Models to Follow Instructions with Human Feedback (InstructGPT).

[61] SDN-Xplain, 2026. SDN-Xplain: LLM-Driven Anomaly Detection and Human-Interpretable Explanation in SDN.

[62] Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN, 2025. Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN.

[63] Hossain, M. K. and others, 2025. NetIntent: Leveraging Large Language Models for End-to-End Intent-Based SDN Automation.

[64] Adanza, D. and others, 2025. Leveraging Generative AI for Intent-Based Networking.

[65] Towards Intent-Based Network Management: Large Language Models for Intent Extraction in 5G Core Networks, 2024. Towards Intent-Based Network Management: Large Language Models for Intent Extraction in 5G Core Networks.

[66] Chen, Y. and others, 2024. NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services.

[67] Exploring LLM-Based Agents for Root Cause Analysis, 2024. Exploring LLM-Based Agents for Root Cause Analysis.

[68] Empowering AIOps: Leveraging Large Language Models for IT Operations Management, 2025. Empowering AIOps: Leveraging Large Language Models for IT Operations Management.

[69] Wang, Z. and others, 2025. Intent-Driven Network Management with Multi-Agent LLMs.

[70] LLM-Driven Multi-Agent Framework for Autonomous Network Slice Management, 2026. LLM-Driven Multi-Agent Framework for Autonomous Network Slice Management.

[71] Dettmers, T. and Pagnoni, A. and Holtzman, A. and Zettlemoyer, L., 2023. QLoRA: Efficient Finetuning of Quantized LLMs.

[72] Zhang, Q. and Chen, M. and Bukharin, A. and others, 2023. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning.

[73] Liu, S.-Y. and Wang, C.-Y. and Yin, H. and others, 2024. DoRA: Weight-Decomposed Low-Rank Adaptation.

[74] Houlsby, N. and Giurgiu, A. and Jastrzebski, S. and others, 2019. Parameter-Efficient Transfer Learning for NLP.

[75] Han, Z. and Gao, C. and Liu, J. and Zhang, J. and Zhang, S. Q., 2024. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey.

[76] Hu, Z. and Wang, L. and Lan, Y. and others, 2023. LLM-Adapters: An Adapter Family for Parameter-Efficient Fine-Tuning of Large Language Models.

[77] Zhang, R. and Han, J. and Liu, C. and Gao, P. and others, 2024. LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention.

[78] Meng, X. and Lin, C. and Wang, Y. and Zhang, Y., 2025. NetGPT: Generative Pretrained Transformer for Network Traffic.

[79] Ginige, Y. and Dahanyayaka, T. and Huang, Y. and Jourjon, G. and Seneviratne, S., 2024. TrafficGPT: An LLM Approach for Open-Set Encrypted Traffic Classification.

[80] Cui, T. and Lin, X. and Li, S. and Chen, M. and others, 2025. TrafficLLM: Enhancing Large Language Models for Network Traffic Analysis with Generic Traffic Representation.

[81] Brown, T. B. and Mann, B. and Ryder, N. and others, 2020. Language Models are Few-Shot Learners.

[82] Achiam, J. and Adler, S. and Agarwal, S. and others, 2023. GPT-4 Technical Report.

[83] Arrieta, A. B. and Diaz-Rodriguez, N. and Del Ser, J. and Bennetot, A. and Tabik, S. and others, 2020. Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI.

[84] Kadavath, S. and Conerly, T. and Askell, A. and others, 2022. Language Models (Mostly) Know What They Know.

[85] Sarker, I. H., 2024. Explainable AI for Cybersecurity Automation, Intelligence and Trustworthiness in Digital Twin: Methods, Taxonomy, Challenges and Prospects.

[86] Danilevsky, M. and Qian, K. and Aharonov, R. and Katsis, Y. and Kawas, B. and Sen, P., 2020. A Survey of the State of Explainable AI for Natural Language Processing.

[87] Ribeiro, M. T. and Singh, S. and Guestrin, C., 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier.

[88] Lundberg, S. M. and Lee, S.-I., 2017. A Unified Approach to Interpreting Model Predictions.

[89] Ali, T. and Kostakos, P., 2023. HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs).

[90] Houssel, N. and Singh, S., 2024. Towards Explainable Network Intrusion Detection using Large Language Models.

[91] Pizarro, I. and others, 2025. X-Flow: Explanation-Augmented Prompting for LLM-Based Intrusion Detection.

[92] Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN, 2025. Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN.

[93] Guo, C. and Pleiss, G. and Sun, Y. and Weinberger, K. Q., 2017. On Calibration of Modern Neural Networks.

[94] Lin, S. and Hilton, J. and Evans, O., 2022. Teaching Models to Express Their Uncertainty in Words.

[95] Zhu, C. and Xu, B. and Wang, Q. and Zhang, Y. and Mao, Z., 2023. On the Calibration of Large Language Models and Alignment.

[96] Moustafa, N. and Slay, J., 2015. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems.

[97] Moustafa, N. and Slay, J., 2016. The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set.

[98] Sharafaldin, I. and Lashkari, A. H. and Ghorbani, A. A., 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization.

[99] Canadian Institute for Cybersecurity (CIC) and Communications Security Establishment (CSE), 2018. CSE-CIC-IDS2018 Dataset.

[100] Ring, M. and Wunderlich, S. and Scheuring, D. and Landes, D. and Hotho, A., 2019. A Survey of Network-Based Intrusion Detection Data Sets.

[101] Tavallaee, M. and Bagheri, E. and Lu, W. and Ghorbani, A. A., 2009. A Detailed Analysis of the KDD CUP 99 Data Set.

[102] Lantz, B. and Heller, B. and McKeown, N., 2010. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks.

[103] Mohammadi, S. and Buzura, S. and others, 2023. A Hybrid Software and Hardware SDN Simulation Testbed.

[104] Elsayed, M. S. and Le-Khac, N.-A. and Dev, S. and Jurcut, A. D., 2020. InSDN: A Novel SDN Intrusion Dataset.

[105] Zhang, Y. and Liu, Q., 2022. On IoT Intrusion Detection Based on Data Augmentation for Enhancing Learning on Unbalanced Samples.

[106] Aceto, G. and Ciuonzo, D. and Montieri, A. and Persico, V. and Pescapa, A., 2024. Synthetic and Privacy-Preserving Traffic Trace Generation Using Generative AI Models for Training Network Intrusion Detection Systems.

[107] Kang, F. and others, 2025. A Class Imbalance Data Augmentation Method for Network Intrusion Detection Based on M2M-VAEGAN.

[108] Tsai, C.-P. and Teng, G. and Wallis, P. and Ding, W., 2025. AnoLLM: Large Language Models for Tabular Anomaly Detection.

[109] Fang, X. and Xu, W. and Tan, Z. and Liu, J. and Huang, Q. and others, 2024. Large Language Models on Tabular Data: A Survey.

[110] Li, A. and Wang, S. and Wang, G. and Zhong, S., 2024. Anomaly Detection of Tabular Data Using LLMs.

[111] Synthetic Network Traffic Data Generation: A Comparative Analysis, 2024. Synthetic Network Traffic Data Generation: A Comparative Analysis.

[112] Sui, Y. and Zhou, M. and Zhou, M. and Han, S. and Zhang, D., 2024. Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study.

[113] Hegselmann, S. and Buendia, A. and Lang, H. and Agrawal, M. and Jiang, X. and Sontag, D., 2023. TabLLM: Few-shot Classification of Tabular Data with Large Language Models.

[114] Wei, J. and Wang, X. and Schuurmans, D. and others, 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

[115] Ding, N. and Qin, Y. and Yang, G. and Wei, F. and others, 2022. Delta Tuning: A Comprehensive Study of Parameter Efficient Methods for Pre-trained Language Models.

[116] Rodrigues, Joao Pedro Santos, 2026. Deteccao e Mitigacao Inteligente de Anomalias em Redes Definidas por Software com Grandes Modelos de Linguagem Open Source.